

# Prize Bond Checking App — 110+ Viva Questions & Answers

---

Short, easy-to-remember answers. Read the bold keywords twice.

---

## A. Basic Project Questions (1–15)

- Q1. What is your project?** A: A cross-platform Flutter app that lets Pakistani prize bond holders instantly check draw results, save a bond portfolio that is auto-checked, receive push notifications when they win, view the official draw schedule, download/generate result PDFs, and buy/sell bonds through a marketplace. It works offline using a local cache.
- Q2. What problem does it solve?** A: People check bond results manually in long PDF lists — slow, error-prone, and many winners never claim prizes. My app automates checking and notifies winners instantly.
- Q3. Who are the users?** A: Normal users (bond holders), guests (browse without account), and an admin who publishes results and manages users.
- Q4. What are prize bonds?** A: Investment bonds issued by Pakistan's National Savings. Instead of interest, bond numbers enter periodic lucky draws with cash prizes. Denominations: Rs. 100, 200, 750, 1500, 7500, 15000, 25000, 40000.
- Q5. What technologies did you use?** A: Flutter (Dart) for the app, GetX for state management, Firebase (Auth, Firestore, Storage, Cloud Messaging, Cloud Functions) as backend, Hive and GetStorage for offline/local storage.
- Q6. Why is this project useful in the real world?** A: Millions own prize bonds; unclaimed prizes are common because checking is manual. Automatic checking + notifications directly solves that.
- Q7. Is your app Android-only?** A: No — Flutter compiles the same codebase to Android and iOS (project also has web/desktop targets configured).
- Q8. What is the main flow of your app?** A: Splash → session check → Sign In (or guest) → Main Screen with 4 tabs (Home, My Bonds, Market, Draws) → Settings/Profile. Admins go Splash → Admin Dashboard → Create Draw / Manage Users.
- Q9. How many user roles exist?** A: Two — `admin` and `normal_user` — stored in the user's Firestore document, plus a guest mode without login.
- Q10. What happens after signup?** A: Account is created with status **pending**; the admin must approve it (set to **active**) before login works. Suspended users are blocked.
- Q11. Which database do you use?** A: Cloud Firestore (cloud, source of truth) plus Hive (offline draw cache) and GetStorage (settings, saved bonds, sync queue) on the device.
- Q12. How does the app work offline?** A: Draws are cached in Hive on every online load; when offline the app reads the cache, so bond checking still works. New bonds and marketplace listings are saved locally first and synced later.
- Q13. What is the denomination in your app?** A: The face value of a bond (Rs. 100 up to Rs. 40,000). Every draw, bond, and marketplace listing belongs to one denomination.
- Q14. Why did you build this as an FYP?** A: It combines a real local problem with a full modern stack: mobile UI, cloud backend, push notifications, offline sync, file handling, and role-based admin — good breadth and practical value.
- Q15. How long is a bond number?** A: Exactly 6 digits — the app validates this everywhere (regex `^\d{6}$` in the scanner, length + numeric checks in forms).
-

## B. Flutter Questions (16–35)

**Q16. Why did you choose Flutter?** A: One codebase for Android and iOS, hot reload for fast development, high performance (compiles to native ARM code, renders with its own engine), a rich widget library, and strong Firebase integration.

**Q17. Why not React Native?** A: React Native uses a JavaScript bridge to native components, which can bottleneck; Flutter compiles ahead-of-time to native code and draws its own UI, giving consistent 60fps and identical look on both platforms. Dart's strong typing also catches errors at compile time.

**Q18. Why not native Android (Kotlin)?** A: Native would need two separate apps for two platforms — double the work for a solo FYP. Flutter gave me both from one codebase.

**Q19. What is a widget?** A: Everything in Flutter's UI is a widget — an immutable description of part of the interface. Widgets nest into a widget tree; Flutter renders the tree efficiently.

**Q20. StatelessWidget vs StatefulWidget?** A: Stateless has no internal mutable state — it rebuilds only when its inputs change. Stateful holds a State object that can change over time via `setState`. In my app most screens are Stateless because state lives in GetX controllers instead.

**Q21. What is hot reload?** A: Injecting updated code into the running Dart VM so UI changes appear in under a second without restarting the app or losing state.

**Q22. What is the widget tree?** A: The hierarchy of nested widgets. Mine: `ScreenUtilInit` → `GetMaterialApp` → `SplashScreen/MainScreen` → `Scaffold` → `IndexedStack` → tab screens → cards/lists.

**Q23. What is `BuildContext`?** A: A handle to a widget's location in the tree, used to find ancestors like Theme or Navigator. GetX reduces my dependence on it (navigation and snackbars work without context).

**Q24. What is a `Scaffold`?** A: The basic Material page skeleton — provides app bar, body, floating action button, bottom navigation bar, snackbars.

**Q25. How is your bottom navigation built?** A: A `BottomNavigationBar` with 4 fixed tabs driving an `IndexedStack` whose index is a reactive `RxInt` in `NavController`. `IndexedStack` keeps all tabs alive, so state and scroll positions survive tab switches.

**Q26. What is `IndexedStack` and why use it?** A: A stack that shows only one child at a time but keeps all children mounted. I use it so tab data doesn't reload on every switch — better UX and fewer Firestore reads.

**Q27. How did you make the UI responsive?** A: `flutter_screenutil` with a 400×812 design size — I write `20.w`, `12.h`, `16.sp` and sizes scale proportionally to any screen.

**Q28. What is `async` / `await`?** A: Dart's way to run long operations (network, disk) without blocking the UI thread. `await` pauses the function, not the app; the UI keeps rendering.

**Q29. What is a `Future` and a `Stream`?** A: A Future is one asynchronous value ("the result will arrive once"); a Stream is many values over time. Firestore queries return Futures; real-time listeners return Streams.

**Q30. How do you navigate between screens?** A: GetX navigation — `Get.to(Screen())` to push, `Get.offAll(Screen())` to replace the whole stack (after login/logout), `Get.back(result: value)` to return data (my scanner returns the scanned number this way).

**Q31. What is `pubspec.yaml`?** A: The project manifest — declares the app name, Dart SDK constraint, all package dependencies with versions, and bundled assets/fonts.

**Q32. What is the difference between `main()` and `runApp()`?** A: `main()` is the Dart entry point where I initialize Firebase, storage, Hive and notifications; `runApp()` inflates the root widget and attaches it to the screen.

**Q33. How do you handle the keyboard/forms?** A: `Form` + `TextFormField` with validators (`AutovalidateMode.onUserInteraction`), input formatters (digits-only, length limits), and a tap-outside handler to hide the keyboard.

**Q34. What are keys in Flutter?** A: Identifiers that help Flutter match widgets between rebuilds (e.g. `GlobalKey<FormState>` for validating my signup form).

**Q35. What theme setup do you use?** A: Material 3 `ThemeData` with a `ColorScheme.fromSeed` on the primary dark-teal color (#1A3C40), themed cards (16px radius), buttons, inputs, and centralized text styles — so the whole app changes from one file.

---

## C. GetX / State Management Questions (36–50)

**Q36. What is state management and why do you need it?** A: Keeping UI in sync with changing data (loading flags, lists, user session). Without it you'd pass data manually through constructors and call `setState` everywhere — unmanageable as the app grows.

**Q37. Why GetX over Provider or BLoC?** A: GetX combines three tools in one small package — reactive state, dependency injection, and context-free navigation — with minimal boilerplate. BLoC is powerful but verbose (events/states/streams for everything); Provider still needs context and separate navigation. For a solo project GetX was the fastest to build correctly.

**Q38. What is `.obs` and `Obx`?** A: `.obs` makes a variable observable (`RxBool`, `RxList` ...). `Obx(() => Text(ctrl.count.value.toString()))` subscribes to whatever Rx values it reads and rebuilds **only that widget** when they change.

**Q39. `Get.put` vs `Get.find` vs `Get.lazyPut`?** A: `Get.put` creates and registers an instance immediately; `Get.find` retrieves an already-registered one; `Get.lazyPut` registers a builder that creates it on first use. I `Get.put` permanent controllers at startup and screen controllers when the screen opens, then `Get.find` them elsewhere.

**Q40. What does `permanent: true` mean?** A: The controller is never deleted from memory when its screen closes. I use it for app-wide controllers like `DrawController`, `AuthController`, and `ConnectivityService`.

**Q41. `Obx` vs `GetBuilder`?** A: `Obx` is reactive — rebuilds automatically when Rx values change. `GetBuilder` is manual — rebuilds when the controller calls `update()`. `GetBuilder` is slightly lighter; I use it in a few forms, `Obx` everywhere else.

**Q42. What is dependency injection?** A: Providing objects from a central registry instead of constructing them inside every class. `Get.put` / `Get.find` is a service locator — controllers are created once and shared, which saves memory and keeps state consistent.

**Q43. What is a `GetxService`?** A: Like a controller but intended to live for the whole app lifetime — my `ConnectivityService` extends it.

**Q44. Controller lifecycle methods?** A: `onInit()` (called when created — I load data here), `onReady()` (after first frame), `onClose()` (cleanup — my `ScannerController` disposes the camera here).

**Q45. How does data flow in your architecture?** A: View fires an event (button tap) → Controller method runs business logic → Service talks to Firestore/Hive → Controller updates Rx state → `Obx` widgets rebuild automatically. One direction, easy to debug.

**Q46. What architecture pattern is this?** A: MVC with a Service layer, implemented with GetX — Models (data classes), Views (screens without logic), Controllers (state + logic), Services (Firestore/Hive/FCM/PDF access).

**Q47. Why a separate Service layer?** A: Separation of concerns — controllers don't know Firestore query syntax; if I swap Firestore for a REST API, only services change. Also makes logic testable.

**Q48. How do two screens share data?** A: Through a shared controller — e.g. Home and Draws both `Get.find<DrawController>()`, so a refresh in one is instantly visible in the other.

**Q49. How does the scanner give its result to the Home screen?** A: `Get.back(result: scannedNumber)` pops the scanner returning the string; Home `await s Get.to(ScannerScreen())` and fills the text field with the result.

**Q50. What is reactive programming?** A: A style where the UI declares what it depends on, and updates flow automatically when data changes — my `.obs` variables + `Obx` widgets are exactly that.

---

## D. Firebase Questions (51–70)

**Q51. What is Firebase?** A: Google's Backend-as-a-Service platform — ready-made authentication, databases, file storage, push notifications, and serverless functions with official Flutter SDKs.

**Q52. Why Firebase instead of your own server?** A: No server to build, host, or secure; free tier fits an FYP; automatic scaling; real-time database; battle-tested auth. I focused my time on app features.

**Q53. What is Cloud Firestore?** A: A NoSQL cloud document database — data lives in collections of documents (JSON-like). It supports real-time listeners, offline persistence, and powerful queries.

**Q54. Which Firestore collections do you have?** A: `customers` (user profiles), `draws` (results), `saved_bonds` (portfolios), `marketplace` (listings), `notifications` (in-app alerts).

**Q55. SQL vs NoSQL — why NoSQL here?** A: My data is naturally document-shaped (a draw with a list of winning numbers) and read-heavy with simple queries; NoSQL scales horizontally and needs no schema migrations. I don't need multi-table joins or transactions across many entities.

**Q56. How does Firebase Authentication work?** A: The app sends email+password to Firebase; Firebase verifies credentials (it stores only password hashes) and returns a user with a unique **UID** and an ID token. That UID keys all the user's data in Firestore.

**Q57. How do you store extra user info Firebase Auth doesn't hold?** A: A profile document in `customers/{uid}` — name, phone, city, role, status, FCM token, bond count.

**Q58. What is FCM and how do notifications work end-to-end?** A: Firebase Cloud Messaging delivers push messages to devices. Flow: app obtains a device **token** at login and saves it in the user's document → admin publishes a draw → the Firestore trigger Cloud Function reads all active users' tokens and sends a broadcast push → FCM routes it to each device → my local-notifications code displays it and, on tap, navigates to the Draws or My Bonds screen.

**Q59. How do winners get their special notification?** A: The same Cloud Function compares the new draw's winning numbers with every `saved_bonds` document of that denomination; each match gets a personal "Congratulations" push plus an in-app notification record, with a duplicate check so nobody is notified twice for the same draw and bond.

**Q60. What is a Cloud Function and which ones do you have?** A: Server-side Node.js code that runs on Google's servers in response to events. Mine: **onNewDraw** (Firestore trigger on new draw → broadcast + winner matching), **onUserSignup** (trigger on new user), **getAdminStats** (secure callable that verifies the caller is an admin before returning counts).

**Q61. Why do winner notifications need a Cloud Function — why not send from the app?** A: Client apps can't (and shouldn't) send pushes to other users — that needs the FCM server key, which must never ship inside an app. Also the sender's phone might be offline; a server function always runs.

**Q62. What is an FCM token and what if it expires?** A: A unique address for one app installation. My code listens to token refresh and updates Firestore; the Cloud Function removes tokens FCM reports as invalid.

**Q63. How does the app show a notification when it is completely closed?** A: A **top-level background handler** registered with `FirebaseMessaging.onBackgroundMessage`, annotated `@pragma('vm:entry-point')` so the Dart runtime can invoke it in a background isolate even when the app process was killed.

**Q64. What is Firebase Storage used for?** A: The admin's official draw-result PDFs, stored at `draw_results/{denomination}/{drawNumber}.pdf`. Users download them with live progress.

**Q65. What are Firestore security rules?** A: Server-enforced access control per collection — e.g. users may only write their own documents, only admins may write draws. They protect data even if someone bypasses the app.

**Q66. What is a Firestore `count()` aggregation and where do you use it?** A: A server-side count that returns only a number instead of downloading documents — I use it for the admin dashboard stats (total users, bonds, draws). Fast and cheap.

**Q67. How do you prevent one user reading another's bonds?** A: Every `saved_bonds` document stores a `userId`; queries filter by the logged-in UID, and security rules enforce it server-side.

**Q68. What happens in Firestore when the admin publishes a draw?** A: One `add()` to `draws` → that single write triggers the notification broadcast, winner matching, bond flagging (`isWinner`, `winningDrawId`), and cache refresh. One write, many effects.

**Q69. What is the difference between `get()` and `snapshots()` in Firestore?** A: `get()` fetches once (Future); `snapshots()` is a real-time Stream that fires on every change. I mainly use `get()` plus pull-to-refresh, and streams for notifications.

**Q70. Does Firestore work offline by itself? Why did you add Hive?** A: Firestore has its own offline cache, but I wanted **explicit control**: guaranteed cache of all draws, custom offline checking logic, a visible sync time, and cache that survives independent of Firestore SDK behavior. Hive gives me deterministic offline reads.

---

## E. Database / Storage Questions (71–78)

**Q71. What is Hive?** A: A fast, lightweight NoSQL database written in pure Dart, storing data in binary "boxes" on the device. No native dependencies, faster than SQLite for key-value/document workloads.

**Q72. Which Hive boxes do you have?** A: `draws_cache` (all draws keyed by id), `saved_bonds_cache`, and `app_settings` (last sync time, local PDF paths).

**Q73. Hive vs GetStorage — why both?** A: Hive for structured, larger datasets (all draws as JSON); GetStorage for tiny synchronous key-value data (toggles, session timestamp, small lists, pending queue). Right tool per job.

**Q74. Why not SQLite?** A: My local data is documents, not relational tables — no joins needed. Hive is simpler, pure Dart, and faster for this workload.

**Q75. How is a bond checked offline?** A: `checkBondOffline(number, denomination)` scans the cached draws of that denomination and returns the winning draw if the number appears in its winning numbers list.

**Q76. When is the cache updated?** A: On every successful online load of draws, after the admin publishes a draw, and the last-sync timestamp is recorded each time.

**Q77. What happens to local data when a different user logs in?** A: `UserController` detects the UID change and wipes the previous user's bonds, caches, and profile — preventing data leakage between accounts on a shared device.

**Q78. How do offline marketplace listings sync?** A: New listings are queued in GetStorage (`pending_marketplace_listings`); on the next online load the queue is flushed to Firestore, and failed items are re-queued for retry.

---

## F. Feature-Specific Questions (79–92)

**Q79. Explain the scanner internally.** A: `mobile_scanner` opens the back camera and runs a barcode detector on the frames (ML Kit on Android, AVFoundation on iOS). On detection I read the barcode's raw string, validate it with regex `^\d{6}$`, stop the camera, and show the number with "Use This Number", which returns it to Home. Invalid codes are silently ignored so scanning continues.

**Q80. Why QR scanning instead of OCR?** A: OCR of printed digits is unreliable on worn paper and needs heavy ML models; QR decoding is deterministic and instant. It was a deliberate reliability trade-off; OCR is future work.

**Q81. What if camera permission is denied?** A: `mobile_scanner`'s error builder shows a dedicated message view instead of crashing; `permission_handler` is included for requesting permissions.

**Q82. How is a result PDF produced when the admin never uploaded one?** A: The `pdf` package builds an A4 document in Dart — header with draw details, winning numbers in a 6-column grid — saves it locally and opens it with `open_filex`. So every draw has a PDF, uploaded or not.

**Q83. How does the PDF download show progress?** A: Firebase Storage's `writeToFile` task streams `snapshotEvents`; I map `bytesTransferred / totalBytes` into a reactive progress map keyed by draw id, and the UI shows a progress bar per card.

**Q84. How is the next draw date calculated?** A: The official schedule rules are encoded per denomination (monthly on the 1st or 15th, bi-monthly, quarterly month sets); the app computes the next occurrence after today and shows a countdown.

**Q85. How does the marketplace WhatsApp contact work?** A: The seller's phone is normalized to international format (+92...), the app builds `https://wa.me/<phone>?text=<message>` and opens it via `url_launcher`; if WhatsApp isn't available, it falls back to a `tel:` call.

**Q86. How are duplicate saved bonds prevented?** A: Locally: a check on number + denomination before adding. In Firestore: a query for the same `userId` + `bondNumber` + `denomination` before insert.

**Q87. What validations run when adding a bond?** A: Signed-in check, exactly 6 characters, numeric only, duplicate check — each failure shows a specific snackbar.



**Q88. What are the three PDF button states on Draw Detail?** A: **Download PDF** (official file exists in Storage), **Generate PDF** (no official file → build locally), **Open PDF** (already on this device — path remembered in the settings box).

**Q89. What do the settings toggles do?** A: Persist `notifications_enabled` and `auto_check_enabled` flags in GetStorage, which gate notification display and automatic bond checking.

**Q90. How does guest mode work?** A: "Continue as Guest" opens mainScreen without a Firebase user; Home and Draws work fully, while My Bonds and Marketplace show sign-in walls (auth is checked via `FirebaseAuth.instance.currentUser == null`).

**Q91. How does the admin approve a user?** A: Manage Users lists all `customers`; tapping one opens a status sheet (active / pending / suspended) that updates the user's Firestore document — login checks that status.

**Q92. Why is the winning-numbers input a comma-separated text field?** A: Fastest for the admin to paste hundreds of numbers at once; the app splits on commas, trims whitespace, and drops empties before saving as a list.

---

## G. Supervisor-Level / Hard Questions (93–105)

**Q93. Why did you choose this architecture?** A: MVC+Services withGetX gives clear separation (views dumb, controllers testable, services swappable), minimal boilerplate for a solo developer, and reactive UI with surgical rebuilds. Clean Architecture would add more layers than a project of this size benefits from.

**Q94. How would you scale this to 1 million users?** A: Firestore and FCM scale automatically; my FCM sends are already chunked (500/batch). Next steps: FCM **topics** instead of token lists for broadcasts, pagination on draw lists, tightened security rules with App Check, and moving winner matching to batched queries with composite indexes.

**Q95. How do you secure user data?** A: Firebase Auth handles credentials; role and status are enforced at login and splash; admin functions verify the caller's role **server-side**; per-user data is filtered by UID and protected by security rules; local caches are wiped on account switch.

**Q96. What was your hardest technical challenge?** A: Reliable end-to-end notifications across app states — foreground, background, and terminated — which needed FCM + a local notification channel + a top-level background handler + tap navigation, and a Cloud Function with token hygiene and duplicate prevention. (Pick this or offline sync.)

**Q97. What happens if the PDF upload fails mid-publish?** A: The admin is asked whether to publish without the PDF; the draw itself is never lost. Downloads verify file integrity and fall back to local generation on failure.

**Q98. How did you test the app?** A: Manual feature testing on real devices and emulator — every flow (auth states, offline mode by toggling airplane mode, scanner with generated QR codes, admin publish → notification arrival), plus Firebase console verification of documents and function logs. Unit tests for controllers are noted as future work.

**Q99. Race condition: what if two users save the same bond?** A: That's allowed by design — multiple people can hold different physical bonds while watching the same number, and each gets their own document and their own winner notification.

**Q100. Why is checking client-side instead of a server query per check?** A: The draws are already on the device (cached for offline), so checking locally is instant, free, and works without internet. Server-side checking would add latency and cost for zero accuracy gain.

**Q101. What is the single source of truth in your app?** A: Firestore. Hive and GetStorage are replicas/queues that always defer to Firestore when online.

**Q102. What would you do differently if you rebuilt it?** A: Define named routes and a route table from day one, write unit tests alongside features, remove legacy prototype files earlier, and add pagination to Firestore queries from the start.

**Q103. How do you handle API/version breaking changes in packages?** A: Versions are pinned with caret ranges in pubspec.yaml and locked exactly in pubspec.lock, so builds are reproducible; upgrades are deliberate.

**Q104. Is your admin panel secure — what stops me calling admin functions?** A: The callable function rejects unauthenticated requests and reads the caller's role from Firestore on the server; a client cannot fake `request.auth` because it comes from the verified Firebase ID token.

**Q105. Why only a 1-hour session window?** A: An app-level control on top of Firebase's token auto-refresh — a simple policy to force re-verification of role/status; easily adjustable.

---

## H. HR-Style Questions (106–112)

**Q106. Summarize your project in 30 seconds.** A: (Use the one-line pitch from the guide — practice it aloud twice.)

**Q107. What did you learn from this project?** A: End-to-end product thinking: designing offline-first sync, integrating five Firebase services, writing my first server-side Cloud Functions, and making real engineering trade-offs like QR-vs-OCR and WhatsApp-vs-in-app-chat.

**Q108. What are you most proud of?** A: The notification pipeline — one admin action fans out to a broadcast push, personal winner alerts, bond flagging and cache refresh, and it works even when users' apps are closed.

**Q109. If given 3 more months, what would you add?** A: Automatic result import from the National Savings source, OCR scanning, Urdu localization, and marketplace payments.

**Q110. Did you copy this from somewhere?** A: The idea addresses a well-known local problem, but the design, architecture, data model, Cloud Functions and every screen were built by me — I can walk through any file and explain or modify it live.

**Q111. Can this be published to the Play Store?** A: Yes — it needs release signing, tightened security rules, a privacy policy, and store assets. Technically the build is store-ready.

**Q112. What's the business model?** A: Free app; potential monetization via marketplace listing fees or ads on the results screen; the public-service value (unclaimed prizes) also suits sponsorship by financial institutions.